

Data Analytics Project 1

Executive Summary

This report documents the data cleaning and preprocessing pipeline implemented in the Jupyter Notebook Code_File_Project1.ipynb. The notebook reads a raw Excel dataset, performs systematic quality checks, applies a series of transformations to resolve data issues and exports a cleaned CSV file ready for downstream analysis.

The pipeline covers six key areas: duplicate detection, missing value imputation, date standardization, whitespace normalization, numeric precision enforcement and data type optimization.

Environment & Libraries

The following Python libraries are imported at the start of the notebook:

Library	Purpose
pandas	Core data manipulation and Data-Frame operations
numpy	Numerical computations
matplotlib / matplotlib.patches	Data visualization and charting
seaborn	Statistical data visualization
warnings	Suppress non-critical runtime warnings
re	Regular expressions for date format validation

Data Ingestion

Loading the Raw Dataset

The raw dataset is loaded from an Excel file using pandas:

```
df_raw = pd.read_excel('C:\\Users\\DELL\\OneDrive\\Desktop\\intern project\\Dataset for Data Analytics.xlsx')
```

A working copy is immediately created to preserve the original data for before/after comparisons:

```
#create copy for before after comparison
df = df_raw.copy()
```

Exploratory Data Assessment

Before applying any transformations, the notebook inspects the dataset using standard pandas diagnostics:

Method	Information Obtained
df.head()	Preview of the first five records
df.info()	Column names, data types and non-null counts
df.duplicated().sum()	Total number of fully duplicated rows
df.describe()	Statistical summary of numeric columns
df.dtypes.to_string()	Explicit dtype listing for all columns
df.isnull().sum()	Count of missing values per column

Data Cleaning Steps

Missing Value Imputation - CouponCode

The `isnull().sum()` check reveals that the `CouponCode` column contains missing values. These are intentionally imputed with the sentinel value 'NONE' to distinguish no-coupon orders from genuinely missing data:

```
#Impute missing CouponCode with 'NONE'
before_null = df['CouponCode'].isnull().sum()
df['CouponCode'] = df['CouponCode'].fillna('NONE')
after_null = df['CouponCode'].isnull().sum()
```

A `value_counts()` check after imputation confirms the distribution of coupon codes, including the new 'NONE' category.

Duplicate Detection

Two independent duplicate checks are run:

- OrderID-level duplicates - identifies OrderIDs that appear more than once, which may indicate duplicate transactions.
- Full-row duplicates - flags records where every column value is identical, indicating true data entry duplication.

```
#Duplicate OrderID check
dup_ids = df[df['OrderID'].duplicated(keep=False)]
print(f"Duplicate OrderIDs found : {df['OrderID'].duplicated().sum()}")
```

Duplicate OrderIDs found : 0

```
#Full row duplicate check
full_dupe_count = df.duplicated().sum()
print(f"Fully duplicated rows found : {full_dupe_count}")
```

Fully duplicated rows found : 0

Date Standardization

The `Date` column is standardized to ISO 8601 format (YYYY-MM-DD) in two steps. First, the column is parsed as datetime and reformatted:

```
df['Date'] = pd.to_datetime(df['Date']).dt.strftime('%Y-%m-%d')
```

Compliance is then verified using a regular-expression pattern match:

```
import re
date_pattern = re.compile(r'^\d{4}-\d{2}-\d{2}$')
non_compliant = df['Date'].apply(lambda x: not bool(date_pattern.match(str(x)))).sum()
print(f"\n Non-compliant dates remaining : {non_compliant}")
```

Whitespace Normalisation

All object-typed (string) columns are iterated and stripped of leading and trailing whitespace using `str.strip()`. The code reports the number of cells corrected in each column:

```
#Strip whitespace from all string columns
str_cols = df.select_dtypes(include=['object']).columns.tolist()
total_fixed = 0

for col in str_cols:
    before = df[col].copy()
    df[col] = df[col].str.strip()
    changed = (before != df[col]).sum()
    if changed > 0:
        print(f" [{col}] whitespace fixed in {changed} cells")
    total_fixed += changed

print(f"\nTotal cells with whitespace corrected: {total_fixed}")
print("All string columns trimmed." if total_fixed == 0 else "Whitespace stripped.")
```

Numeric Precision Enforcement

The `UnitPrice` and `TotalPrice` columns are rounded to two decimal places to ensure consistent monetary representation:

```
#Enforce 2 decimal places on numeric price columns
for col in ['UnitPrice', 'TotalPrice']:
    df[col] = df[col].round(2)
```

Negative Value Check

A data integrity guard confirms that `Quantity`, `UnitPrice`, and `TotalPrice` contain no negative values, which would indicate corrupted or erroneous records:

```
#Negative value check
neg_qty    = (df['Quantity'] < 0).sum()
neg_price  = (df['UnitPrice'] < 0).sum()
neg_total  = (df['TotalPrice'] < 0).sum()
```

If all three counts equal zero, the pipeline logs 'No negative numeric values - data integrity confirmed'.

Data Type Optimization

After cleaning, selected low-cardinality string columns are converted to the pandas category dtype to reduce memory usage and improve query performance:

```
for col in ['Product', 'PaymentMethod', 'OrderStatus', 'ReferralSource', 'CouponCode']:
    df[col] = df[col].astype('category')
```

`df.info()` is called after each major type change to confirm that dtypes are updated as expected.

Output

The cleaned Data-Frame is exported to a CSV file for downstream analysis or reporting:

```
OUTPUT_CSV = "C:\\Users\\DELL\\OneDrive\\Desktop\\intern project\\cleaned_data_project1.csv"
df.to_csv(OUTPUT_CSV, index=False)
```

The index=False argument ensures that the pandas row index is not written as a column in the output file.

Pipeline Summary

The table below summarizes all cleaning steps applied in sequence:

	Step	Column(s) Affected	Action
1	Missing Value Imputation	CouponCode	Fill NaN with 'NONE'
2	Duplicate Detection	OrderID, All columns	Count & report duplicates
3	Date Standardisation	Date	Parse & reformat to YYYY-MM-DD; validate via regex
4	Whitespace Normalisation	All string columns	Strip leading/trailing spaces
5	Numeric Precision	UnitPrice, TotalPrice	Round to 2 decimal places
6	Negative Value Guard	Quantity, UnitPrice, TotalPrice	Assert no negative values
7	Dtype Optimisation	Product, PaymentMethod, OrderStatus, ReferralSource, CouponCode	Cast to category dtype
8	Export	All columns	Write cleaned CSV to disk